

Compiler Design LAB (CIC-351)

Practical File

Faculty Name : Ms. Sakshi Jha

Student Name : Yash Pandey

Roll No : 12214802722 (C-6)

Semester : 5th Semester

Group : 5C6 (CSE Shift-1)



Maharaja Agrasen Institute of Technology, PSP Area,

Sector – 22, Rohini, New Delhi - 110086



Maharaja Agrasen Institute of Technology

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISSION

M1 To lead in the advancement of computer science and engineering through internationally recognized research and education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving and communication skills as an integral component of the profession.

M4 To impart knowledge, skills and cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement.

Department of Computer Science & Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISSION

M1 To lead in the advancement of computer science and engineering through internationally recognized research and education.

M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving and communication skills as an integral component of the profession.

M4 To impart knowledge, skills and cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement



Lab Assessment Sheet

S.No	Experiment	Date	Marks					Total Marks	Signature
			R1	R2	R3	R4	R5		
<u>1.</u>	Basic Program To Practice lex, Bison and lex.yg-c	02-08-24							
<u>2.</u>	WAP To Check whether string Includes keyword	02-08-24							
<u>3.</u>	WAP To Check whether a string belongs to Grammar S → aT, T → bTb	02-08-24							
<u>4.</u>	WAP To Remove left Recursion from Grammar	09-08-24							
<u>5.</u>	WAP To Remove left Factoring from Grammar	16-08-24							
<u>6.</u>	WAP To Design Simple Arithmetic Expressions	16-08-24							
<u>7.</u>	WAP To Design Arithmetic using LEX + YACC	23-08-24							
<u>8.</u>	WAP To design Recursive descent Parser	30-08-24							
<u>9.</u>	WAP To Show all operations On Stack	30-08-24							

Lab Assessment Sheet

[illegible]

Lab Assessment Sheet

[illegible]

Lab Assessment Sheet

[illegible]

Program-1.

Aim :- Basic Program To Practice Flex , Bison & lex.yy.c Working?

Flex : A Flex application is used to **create .l files** (lexical analyzer programs) that define lexical rules for text pattern matching. After writing a .l file, it is executed using the Flex tool, which generates a C program that compiles into an executable, transforming input text based on defined patterns.

Bison : A Bison application complements Flex by **handling grammar and parsing for .l files**. After Flex generates a lexical analyzer, Bison creates a parser that processes the tokenized input from the .l file. Together, they build a complete compiler or interpreter, enabling complex syntax analysis and execution of programming languages or custom commands.

Dev C++ : Dev C++ is an IDE that helps in **executing .l files generated by Flex**. After Flex processes the .l file, it produces a C file. Dev C++ can then be used to compile this C file into an executable, allowing the lexical analyzer to run and process input based on the defined patterns. Dev C++ also provides debugging tools, enabling you to identify and fix errors in the generated C code. It offers an integrated environment to streamline the compilation process, making it easier to execute and test the .l file's output efficiently.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

lex.yy.c : **lex.yy.c is the C source code file generated by the Flex** (or Lex) tool from a '.l' file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

yywrap() : The 'yywrap()' function in a '.l' (Lex) file is a routine that Lex calls when it reaches the end of the input file. By default, it returns 1, signaling the end of input. If 'yywrap()' returns 0, Lex continues scanning with the same rules on more input. This function is useful when dealing with multiple input files or when you want to reset the input state. If your program doesn't require special handling at the end of input, you can simply omit it, as the default behavior is often sufficient.

Source Code

```
%{           #include<stdio.h>           %}

%%

[0-9]       {printf("Welcome");  exit(0);}

.           {printf("Wrong");    exit(0);}

%%
```



```
int yywrap() {      return 1;      }

int main() {
    printf("Enter:");
    yylex();      return 0;
}
```

OUTPUT : -

```
Microsoft Windows [Version 10.0.22631.4037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>flex first.l
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>gcc lex.yy.c
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>a
Enter:29
Welcome
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>a
Enter:Yash
Wrong
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>a
Enter: .
Wrong
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>|
```

Viva Questions(Program-1)

Question 1:

What is LEX?

Answer:

LEX is a lexical analyzer generator that transforms a set of regular expressions into a C program that performs lexical analysis.

Question 2:

What is YACC?

Answer:

YACC stands for "Yet Another Compiler Compiler." It generates a parser that converts a sequence of tokens into a syntax tree according to a grammar.

Question 3:

How do LEX and YACC work together?

Answer:

LEX is used to tokenize the input, while YACC processes these tokens to analyze the structure using a specified grammar.

Question 4:

What is a token in LEX?

Answer:

A token is a categorized block of text, such as keywords, identifiers, or operators, recognized during lexical analysis.

Question 5:

Can YACC handle ambiguous grammars?

Answer:

Yes, but it requires additional techniques such as precedence rules or associativity to resolve conflicts.

⇒ Viva Questions of Exp-01.

Q → What are LEX and YACC Tools?

→ lex is an Computer Program That Generates Lexical Analyzers. It Tokenizes The I/P Source Code Into Meaningful Units called Tokens. The LEX Source Program defines how Tokens are been Recognized & Processed.

→ YACC or "Yet Another Compiler" is an Tool That Generates a Parser for an Context Free Grammar (CFG). It is Impose on I/P To a Comp. Program.

Q → What is The Structure of LEX Program?

→ % {

// C-Declaration & Includes....

% }

declarations

% %

// Translation Rules.

% %

// User - Sub-Routines

→ Defines Modes and Regular Expression Patterns.....

→ Associates Regular Expressions with Patterns & Frameworks.....

→ Contains The user defined C-Code.....

Q → Explain The Structure of YACC Program?

→ The YACC Program is Consist of 3 Section: Declarations, Grammar Rules and Action Statements.

1. Declarations : This Section defines datatypes, Tokens and other Informations.
2. Grammar Rules : These Rules Specify The syntax of The language Using CFG.
3. Action Statements : The statements define actions to be Taken When The Specific Conditions rules are been Recognised Using Parsing.

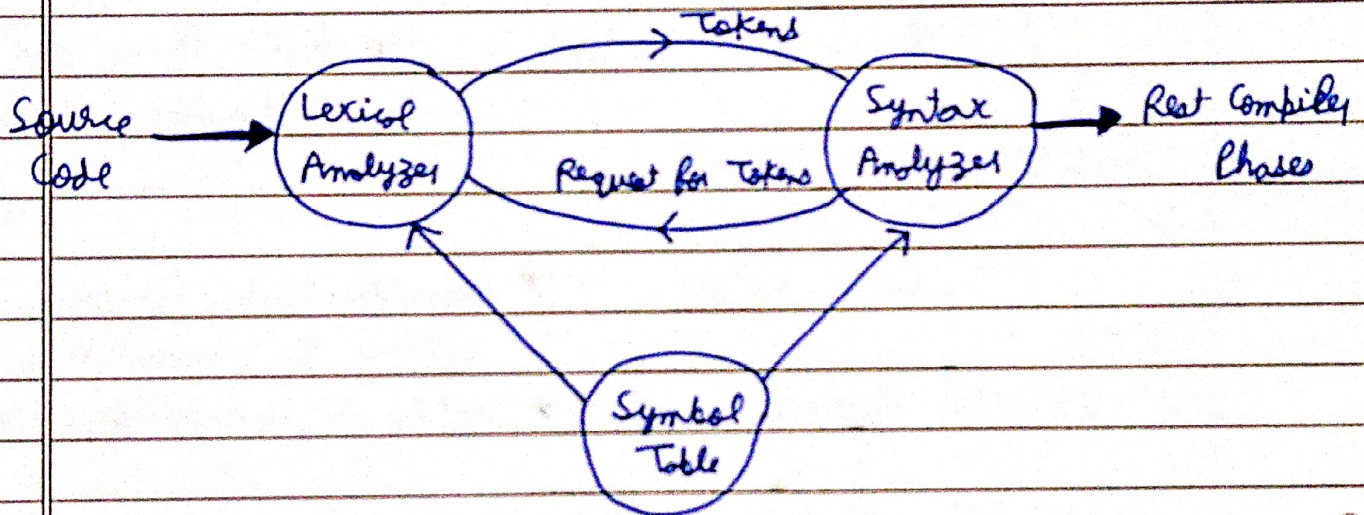
"P.T.O"

Q → What are Tokens or Terminal Symbols?

→ Tokens are been defined as The Smallest Individual Element That Holds The Significance To The Compiler's functioning. There are 5 Type of Tokens In C-Prog. as Keywords, Identifiers, Operators, Special Symbols and Constants.

Q → What Is Lexical Analyzer?

→ The Lexical Analyzers are also known as Lexer or Scanners. It is a Program That Breaks The I/P Into Tokens and Categorizes Them Into Pre-Defined Groups.



Q → What are I/P Buffering Scheme In Lexical Analyzer?

→ The Input Buffering Scheme is an Technique Used In Compiler design to store an Portion of The Source Code In The Memory Before Processing It. It is been Especially Important during The Lexical Analyzer Phase for Compilation which is when The Compiler Reads The Source Code Character By Character by Identifying Tokens.

Program-2.

Aim :- WAP To Check Whether A String Includes Keywords Or Not ?

Keywords : There Are 32 Reserved Keywords In C-Progg. , keywords are reserved words that have special meanings in the language and cannot be used as identifiers (such as variable names, function names, etc.). These keywords form the core of C programming and are used to define various aspects of the language's syntax and behavior. They are fundamental to writing C code and should be used as defined by the language specifications.

The 32 Keywords Are :- auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile & while.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

lex.yy.c : **lex.yy.c is the C source code file generated by the Flex** (or Lex) tool from a '.l' file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

yywrap() : The 'yywrap()' function in a Lex file signals the end of input processing. It is called when 'yylex()' reaches the end of input. By default, 'yywrap()' returns 1 to indicate that no more input files are available. If you need to process multiple files, 'yywrap()' can be customized to handle additional files or perform cleanup tasks. In most simple applications, returning 1 is sufficient to indicate that no further processing is required.

'%%' : In a Lex file, '%%' is used as a delimiter to separate different sections:

- 1. Declarations Section :** Enclosed in '%{ %}', this area includes C code like header files and global variables.
 - 2. Rules Section :** The first '%%' separates this section where you define patterns (regular expressions) and their actions.
 - 3. User Subroutines Section :** The second '%%' marks the end of the rules section and is where you place additional C functions, such as 'yywrap()' and other helper functions.
- Using these delimiters correctly ensures that Lex can properly parse and generate the necessary code for lexical analysis.

Source Code

```
%{
#include <stdio.h>
#include <string.h>

%}

%%

if          { printf("%s' Is A Keyword\n", yytext); }
else        { printf("%s' Is A Keyword\n", yytext); }
continue    { printf("%s' Is A Keyword\n", yytext); }
break       { printf("%s' Is A Keyword\n", yytext); }
char        { printf("%s' Is A Keyword\n", yytext); }
int          { printf("%s' Is A Keyword\n", yytext); }
float        { printf("%s' Is A Keyword\n", yytext); }
double       { printf("%s' Is A Keyword\n", yytext); }
for          { printf("%s' Is A Keyword\n", yytext); }
while        { printf("%s' Is A Keyword\n", yytext); }
return       { printf("%s' Is A Keyword\n", yytext); }
void         { printf("%s' Is A Keyword\n", yytext); }
[0-9]+       { printf("%s' Is A Number\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("%s' Is A Word\n", yytext); }
.|\n        { ; }

%%

int yywrap() { return 1; }

int main() {
    printf("Enter : ");
    yylex();
    return 0;
}
```


OUTPUT :-

```
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122>flex second.1
```

```
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122>gcc lex.yy.c
```

```
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122>a
```

```
if else Yash for while void Pandey 2907 int float
```

```
'if' Is A Keyword
```

```
'else' Is A Keyword
```

```
'Yash' Is A Word
```

```
'for' Is A Keyword
```

```
'while' Is A Keyword
```

```
'void' Is A Keyword
```

```
'Pandey' Is A Word
```

```
'2907' Is A Number
```

```
'int' Is A Keyword
```

```
'float' Is A Keyword
```

Viva Questions(Program-2)

Question 1:

What is a grammar in compiler design?

Answer:

A grammar is a set of production rules that describe the syntactical structure of a language.

Question 2:

What are the components of a grammar?

Answer:

A grammar consists of terminals, non-terminals, a start symbol, and production rules.

Question 3:

How does one check if a string belongs to a grammar?

Answer:

By parsing the string according to the grammar's production rules using techniques like top-down or bottom-up parsing.

Question 4:

What is top-down parsing?

Answer:

Top-down parsing starts from the start symbol and attempts to derive the string by applying production rules.

Question 5:

What role do non-terminals play in a grammar?

Answer:

Non-terminals represent intermediate symbols that can be replaced by terminals or other non-terminals according to the production rules.

Program-3.

Aim :- WAP To Count No. Of Tokens In A String?

Tokens :- In C language, **tokens are the smallest units of code** with meaning. They include keywords , identifiers , literals , operators , and punctuation . Tokens are essential for constructing valid C statements and expressions.

Tokens in C are crucial for syntax analysis during compilation. They represent distinct elements of code, enabling the compiler to understand and process the structure, operations, and data within the program effectively.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

lex.yy.c : **lex.yy.c is the C source code file generated by the Flex** (or Lex) tool from a '.l' file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

yywrap() : The 'yywrap()' function in a Lex file signals the end of input processing. It is called when 'yylex()' reaches the end of input. By default, 'yywrap()' returns 1 to indicate that no more input files are available. If you need to process multiple files, 'yywrap()' can be customized to handle additional files or perform cleanup tasks. In most simple applications, returning 1 is sufficient to indicate that no further processing is required.

Source Code

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
  
int tokens = 0;  
%}  
%%  
[a-zA-Z_][a-zA-Z0-9_]*      { tokens++; }  
[0-9]+                      { tokens++; }  
[ \t\n]+                    { ; }  
.  
%%
```

```
int yywrap() {  
    return 1;  
}  
  
int main() {  
    printf("Enter : ");  
    yylex();  
    printf("Number of tokens: %d\n", tokens);  
    return 0;  
}
```

OUTPUT : -

```
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>flex third.l
```

```
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>gcc lex.yy.c
```

```
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>a
```

```
Enter : Hi, I'm Yash Pandey, a 3rd-year CSE student at MAIT College. Passionate about tech and eager to learn more.
```

```
Number of tokens: 22
```

```
C:\Users\ISHA PANDEY\OneDrive\Desktop\YashPandey122>|
```

Program-3.

Aim :- WAP To Check if a String belongs to a Grammar S->aT T->bT|ε?

In compilers, grammar refers to a set of formal rules that define the syntax of a programming language. It specifies how tokens (the smallest units of meaning, such as keywords, identifiers, operators, and symbols) can be combined to form valid statements, expressions, and program structures.

There are several types of grammar used in compiler design, including:

2. **Context-Free Grammar (CFG):** The most common type, which uses production rules to describe how non-terminal symbols can be replaced by sequences of terminal and non-terminal symbols.
3. **Backus-Naur Form (BNF):** A notation used to express context-free grammars, making it easier to define the syntax of programming languages.
4. **Extended Backus-Naur Form (EBNF):** An extension of BNF that includes additional syntax for expressing repetitions and optional elements, making grammars more concise.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

lex.yy.c : **lex.yy.c is the C source code file generated by the Flex (or Lex) tool** from a '.l' file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

yywrap() : The 'yywrap()' function in a Lex file signals the end of input processing. It is called when 'yylex()' reaches the end of input. By default, 'yywrap()' returns 1 to indicate that no more input files are available. If you need to process multiple files, 'yywrap()' can be customized to handle additional files or perform cleanup tasks. In most simple applications, returning 1 is sufficient to indicate that no further processing is required.

'%%' : In a Lex file, '%%' is used as a delimiter to separate different sections:

1. **Declarations Section :** Enclosed in '%{ %}', this area includes C code like header files and global variables.
2. **Rules Section :** The first '%%' separates this section where you define patterns (regular expressions) and their actions.
3. **User Subroutines Section :** The second '%%' marks the end of the rules section and is where you place additional C functions, such as 'yywrap()' and other helper functions.

1. Gram.l File

```
%{  
#include "y.tab.h"  
%}  
%%  
  
a      return A;  
b      return B;  
.\n    return yytext[0];  
%%  
  
int yywrap(){      return 1;      }
```

2. Gram.y File

```
%{  
#include<stdio.h>  
  
int valid=1;  
%}  
  
% token A B  
%%  
  
str:S'\n' |{  
    printf("Parsed with rule ab^n");  
    return 0;  
}  
  
S : A T ;  
T : B T ;  
%%  
  
int yyerror(char * msg) {  
    valid= 0;  
    return 0;  
}
```

```
int main(){  
    printf("Enter the string to check: \n");  
    yyparse();  
    if(valid == 1){        printf("Valid \n");    }  
    else{        printf("Invalid \n");    }  
    return 0;  
}
```

OUTPUT : -

```
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> flex gram.l  
  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> bison -dy gram.y  
  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> gcc lex.yy.c y.tab.c  
  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> ./a.out  
Enter the string to check: abbbbbb  
Valid  
  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> ./a.out  
Enter the string to check: aaaaaab  
Invalid
```

Viva Questions(Program-3)

Question 1:

What is a keyword in a programming language?

Answer:

A keyword is a reserved word in a programming language that has a predefined meaning and cannot be used as an identifier.

Question 2:

How can a string be checked for keywords?

Answer:

By comparing each word in the string with a list of reserved keywords.

Question 3:

Why is keyword recognition important in compilers?

Answer:

Keywords often represent control structures and data types, so recognizing them is essential for correct syntax analysis.

Question 4:

Can a keyword be used as a variable name?

Answer:

No, keywords are reserved and cannot be used as identifiers in the source code.

Question 5:

What data structure is typically used to store keywords for comparison?

Answer:

Hash tables or arrays are commonly used for fast lookup of keywords.

Program-4.

Aim :- WAP To Remove Left Recursion From An Grammar ?

In compilers, Left recursion occurs in a grammar when a non-terminal in a production rule refers to itself as the first symbol on the right-hand side, leading to potential infinite recursion during parsing. For example, in the production $A \rightarrow A\alpha \mid \beta$, the non-terminal 'A' appears as the leftmost symbol in its own production rule.

This can cause issues with top-down parsers like recursive descent, as they may fall into an infinite loop when attempting to expand left-recursive rules. To avoid this, left recursion must be eliminated, often by rewriting the grammar using techniques such as factoring or introducing new non-terminals.

Problem With Left Recursion → Left recursion is problematic in top-down parsers, particularly recursive descent parsers, because it leads to infinite recursion. In left-recursive grammars, a non-terminal symbol calls itself as the first step in its production, causing the parser to repeatedly try to expand that non-terminal without making progress.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

lex.yy.c : **lex.yy.c is the C source code file generated by the Flex (or Lex) tool** from a '.l' file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

Source Code

```
#include <iostream>

using namespace std;

void removeLeftRecursion(string nonTerminal, vector<string> productions) {
    vector<string> alpha , beta;
    string newNonTerminal = nonTerminal + ""';
    for (const auto& production : productions) {
        if (production[0] == nonTerminal[0]) {
            alpha.push_back(production.substr(1));
        }
    }
    for (const auto& prod : beta) {      cout << prod << " ";    }
```

```

        cout << newNonTerminal << " -> ";
        for (const auto& prod : alpha) {
            cout << prod << " " << newNonTerminal << " | ";
        } cout << "\u03B5" << endl;
    }
}

int main() {
    vector<string> productionsE = {"E+T", "T"};
    vector<string> productionsT = {"T*F", "F"};
    cout << "Original Grammar:" << endl;
    cout << "E -> E + T | T" << endl;
    cout << "T -> T * F | F" << endl;
    cout << "F -> id" << endl << endl;
    cout << "Removing Left Recursion for E:" << endl;
    removeLeftRecursion("E", productionsE);
    cout << endl << "Removing Left Recursion for T:" << endl;
    removeLeftRecursion("T", productionsT);
    cout << endl << "F -> id" << endl;
    return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\DAA\" ; if ($?) { g++ LR.cpp -o LR } ; if ($?) { .\LR }
Original Grammar:
E -> E + T | T
T -> T * F | F
F -> id

Removing Left Recursion for E:
E -> T E'
E' -> +T E' |

Removing Left Recursion for T:
T -> F T'
T' -> *F T' |

F -> id

```

Viva Questions(Program-4)

Question 1:

What is left recursion?

Answer:

Left recursion occurs when a production rule has the form $A \rightarrow A\alpha$, where A is a non-terminal and α is a string of terminals and non-terminals.

Question 2:

Why is left recursion a problem in parsers?

Answer:

Left recursion can cause infinite recursion in top-down parsers, leading to non-termination.

Question 3:

How do you eliminate left recursion?

Answer:

By transforming the grammar into an equivalent one without left recursion, typically by introducing new production rules.

Question 4:

What is an indirect left recursion?

Answer:

Indirect left recursion occurs when a non-terminal indirectly recurses through another non-terminal, such as $A \rightarrow B\alpha$ and $B \rightarrow A\beta$.

Question 5:

Can all left-recursive grammars be converted to non-left-recursive grammars?

Answer:

Yes, all left-recursive grammars can be rewritten to eliminate left recursion.

Program-5.

Aim :- WAP To Remove Left Factoring From An Grammar ?

In compilers, Left factoring is a grammar transformation technique used to eliminate ambiguity and prepare a grammar for top-down parsing, particularly in cases where two or more production rules for a non-terminal start with the same prefix. When this happens, the parser may have difficulty deciding which rule to apply.

Left factoring resolves this by extracting the common prefix and factoring it out, creating a new non-terminal for the remaining parts. For example, if a non-terminal has rules like ' $A \rightarrow \alpha\beta \mid \alpha\gamma$ ', left factoring would rewrite this as ' $A \rightarrow \alpha A'$ ' and ' $A' \rightarrow \beta \mid \gamma$ '. This transformation ensures that the parser can make a decision by reading only one symbol at a time, making the grammar more suitable for predictive parsers.

Problem With Left Factoring → The main problem with left factoring is that while it simplifies grammars for top-down parsers by **eliminating ambiguity**, it can lead to a more complex and less intuitive grammar. By factoring out common prefixes, the new grammar introduces additional non-terminals and productions, making it hard to interpret & maintain.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

lex.yy.c : **lex.yy.c is the C source code file generated by the Flex (or Lex) tool** from a '.l' file. When you run Flex on a '.l' file, it translates the lexical rules and patterns defined in that file into a C program, which is saved as 'lex.yy.c'. This file contains the implementation of a lexical analyzer that processes input text, identifies tokens based on the defined patterns, and can be compiled into an executable program.

Source Code

```
#include <iostream>

#include <vector>

#include <string>

#include <unordered_map>

using namespace std;

void leftFactoring(const string& nonTerminal, const vector<string>& productions) {
    unordered_map<string, vector<string>> prefixMap;
    for (const auto& production : productions) {
        string prefix = production.substr(0, 1);
        prefixMap[prefix].push_back(production.substr(1));
    }
}
```



```

if (prefixMap.size() > 1) {
    cout << nonTerminal << " -> ";
    for (const auto& [prefix, suffixes] : prefixMap) {
        if (suffixes.size() > 1) {
            cout << prefix << nonTerminal + " | ";
        }
    }
    Cout<<endl;
    cout << nonTerminal + " -> ";
    for (const auto& [prefix, suffixes] : prefixMap) {
        if (suffixes.size() > 1) {
            for (const auto& suffix : suffixes) {
                cout << suffix << " | ";
            }
        }
        else {
            cout << suffixes[0] << " | ";
        }
    }
    cout << "ε" << endl;
}
else {
    cout << nonTerminal << " -> ";
    for (const auto& production : productions) {
        cout << production << " | ";
    }
    cout << endl;
}
}

```

```

int main() {
    vector<string> productionsA = {"aB", "aC", "bD", "aE"};
    cout << "Original Grammar:" << endl;
    cout << "A -> aB | aC | bD | aE" << endl;
    cout << "After Left Factoring:" << endl;
    leftFactoring("A", productionsA);
    return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS
Code\Compiler Design\" ; if ($?) { g++ LF.cpp -o LF } ; if ($?) { .\LF }
Original Grammar:
A -> aB | aC | bD | aE
After Left Factoring:
A -> aA' |
A' -> D | B | C | E |

```

Viva Questions(Program-5)

Question 1:

What is left factoring?

Answer:

Left factoring is a technique used to remove ambiguity by factoring out the common prefixes of the production rules in a grammar.

Question 2:

Why is left factoring needed?

Answer:

It simplifies the parsing process by reducing the number of choices at each step, particularly in top-down parsers.

Question 3:

How do you perform left factoring on a grammar?

Answer:

By extracting the common prefixes from the production rules and introducing new non-terminals.

Question 4:

What is the result of applying left factoring?

Answer:

The grammar becomes easier to parse, reducing ambiguity and improving efficiency.

Question 5:

Can left factoring be applied to any grammar?

Answer:

Yes, left factoring can be applied to any grammar that has common prefixes in its production rules.

Program-6.

Aim :- Write an YACC program that parses simple arithmetic expressions?

The YACC and Lex code provided parses and evaluates simple arithmetic expressions involving addition, subtraction, multiplication, and division with integer operands. The Lex part of the program tokenizes numbers and operators, while YACC defines the grammar and precedence rules for parsing the expressions. Operators '+', '-', '*', and '/' are evaluated based on their precedence and associativity.

Parentheses are supported to group expressions, and division by zero is specifically handled to prevent runtime errors. When an expression is successfully parsed, the result of the evaluation is printed to the console. This combination of Lex and YACC forms a basic parser and interpreter for arithmetic expressions.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

1. Arith.y File

```
%{  
#include <stdio.h>  
int yylex(void);  
void yyerror(const char *s);  
%}  
%token NUMBER  
%left '+' '-'      %left '*' '/'  
%%  
expr:  expr '+' expr { printf("%d + %d = %d\n", $1, $3, $1 + $3); }  
      | expr '-' expr { printf("%d - %d = %d\n", $1, $3, $1 - $3); }  
      | expr '*' expr { printf("%d * %d = %d\n", $1, $3, $1 * $3); }  
      | expr '/' expr {  
          if($3 == 0) { yyerror("Division by zero!"); }  
          else { printf("%d / %d = %d\n", $1, $3, $1 / $3); }  
      }
```



```

| '(' expr ')' { $$ = $2; }
| NUMBER      { $$ = $1; }
%%

void yyerror(const char *s) { fprintf(stderr, "Error: %s\n", s); }

int main(void) {
    printf("Enter the expression: ");
    return yyparse();
}

```

2. Arith.1 File

```

%{
#include "y.tab.h"
%}
%%

[0-9]+ { yylval = atoi(yytext); return NUMBER; }
[ \t ]+ /* Ignore whitespace */
\n      { return 0; } /* End of input */
.        { return yytext[0]; }
%%

int yywrap(void) { return 1; }

```

OUTPUT : -

```

C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> flex Airth.1
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> bison -dy Airth.y
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> gcc lex.yy.c y.tab.c
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> ./a.out
Enter the Expression: 3+5*2
5 * 2 = 10
3 + 10 = 13

```

Viva Questions(Program-6)

Question 1:

What is the purpose of YACC in parsing arithmetic expressions?

Answer:

YACC generates a parser that builds a syntax tree for arithmetic expressions based on grammar rules.

Question 2:

What are the precedence rules in YACC?

Answer:

Precedence rules in YACC determine the order of evaluation for operators, helping resolve conflicts between operators like + and *.

Question 3:

How do you define grammar rules in YACC?

Answer:

Grammar rules in YACC are defined in the %% section, specifying how tokens are combined to form valid expressions.

Question 4:

What is an action in a YACC rule?

Answer:

An action is a block of C code associated with a grammar rule that is executed when the rule is matched during parsing.

Question 5:

Can YACC handle both integer and floating-point arithmetic?

Answer:

Yes, YACC can handle multiple types of arithmetic by defining appropriate tokens and actions.

Program-7.

Aim :- WAP To combine LEX and YACC to create a calculator that can evaluate arithmetic expressions with integers.?

LEX and **YACC** are powerful tools commonly used in compiler construction and the development of language parsers. Lex, a lexical analyzer generator, processes input text and converts it into a sequence of tokens, which are the basic building blocks of a programming language. It utilizes regular expressions to define patterns for token recognition. Yacc, which stands for Yet Another Compiler Compiler, takes these tokens and generates a syntax tree based on a defined grammar, typically using context-free grammar rules.

Together, Lex and Yacc facilitate the creation of efficient parsers that can interpret and process complex programming languages, enabling developers to build compilers, interpreters, and other language-processing tools with relative ease. Their combination allows for seamless transition from lexical analysis to parsing, making them a foundational duo in the field of programming language development.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

1. Calc.l File

```
%{  
#include "y.tab.h"  
%}  
%%  
[0-9]+      { yyval = atoi(yytext); return NUMBER; }  
[\n]        { return 0; }  
"+"         { return PLUS; }  
"-"         { return MINUS; }  
"*"         { return MULT; }  
"/"         { return DIV; }  
%%  
int yywrap() {      return 1;      }
```

2. Calc.y File

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
%token NUMBER  
%left PLUS MINUS      %left MULT DIV  
%%  
expr: expr PLUS term { printf("%d\n", $1 + $3); }  
    | expr MINUS term { printf("%d\n", $1 - $3); }  
    | term;          term:      term MULT factor { printf("%d\n", $1 * $3); }  
                                | term          DIV factor { printf("%d\n", $1 / $3); }  
                                | factor;        factor: NUMBER;  
%%  
int main() {  
    printf("\nEnter Any Arithmetic Expression which can have operations Addition,  
                                                Subtraction, Multiplication, Division:\n");  
    yyparse();      return 0;  
}  
int yyerror() { printf("Error\n"); }
```

OUTPUT : -

```
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> flex calc.1  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> bison -dy calc.y  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> gcc lex.yy.c y.tab.c  
C:\Users\ISHA PANDEY OneDrive\Desktop\YashPandey122> ./a.out  
Enter Any Arithmetic Expression which can have operations Addition,  
Subtraction, Multiplication, Division: 4+5  
9  
C:\Users\HP\Desktop\VS Code\Compiler Design>a.exe  
Enter Any Arithmetic Expression which can have operations Addition,  
Subtraction, Multiplication, Division: 4*10  
40
```


Viva Questions(Program-7)

Question 1:

How do LEX and YACC interact in a calculator program?

Answer:

LEX scans the input to tokenize numbers and operators, and YACC parses the tokens and evaluates the expression.

Question 2:

What are shift and reduce operations in YACC?

Answer:

“Shift” moves a token onto the stack, while “reduce” replaces a sequence of tokens with a non-terminal according to a production rule.

Question 3:

How do you handle errors in a LEX/YACC calculator?

Answer:

You can define an error rule in YACC and use `yyerror()` to handle syntax errors and provide feedback.

Question 4:

What is a semantic action in YACC?

Answer:

A semantic action is C code embedded in a YACC rule that performs calculations or stores values during parsing.

Question 5:

What type of arithmetic expressions can this calculator handle?

Answer:

It can handle basic expressions involving addition, subtraction, multiplication, and division.

Program-8.

Aim :- WAP To construct a recursive descent parser for an expression ?

A recursive descent parser is a top-down parser that processes the input based on a set of recursive procedures (or functions). Each procedure corresponds to a non-terminal in the grammar, and the parser tries to match the input with the grammar's production rules by calling these procedures recursively. They read input character by character, consuming terminals as they match against rules, and recursively invoking functions for non-terminals.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <iostream>
#include <string>
using namespace std;
#define SUCCESS 1
#define FAILED 0
int E(), Edash(), T(), Tdash(), F();
const char *cursor;      string input;
int main() {
    cout << "Enter the string: ";
    cin >> input; // Read input from the user
    cursor = input.c_str(); // Initialize cursor to point to the input string
    cout << "\nInput\t\t Action\n";
    cout << "-----\n";
    if(E() && *cursor == '\0') {
        cout << "-----\n";
        cout << "String is successfully parsed\n";
        return 0;
    }
```

```

    } else {
        cout << "-----\n";
        cout << "Error in parsing String\n";
        return 1;
    }
}

int E() {
    cout << cursor << "\t\t E -> T E\n";
    if(T()) {
        if(Edash())
            return SUCCESS;
        else
            return FAILED;
    } else
        return FAILED;
}

int Edash() {
    if(*cursor == '+') {
        cout << cursor << "\t\t E' -> + T E\n";
        cursor++;
        if(T()) {
            if(Edash())
                return SUCCESS;
            else {
                return FAILED;
            }
        } else {
            cout << cursor << "\t\t E' -> $\n";
            return SUCCESS; // E' ->  $\epsilon$  (empty string)
        }
    }
}

```

```

int T() {
    cout << cursor << "\t\t T -> F T\n";
    if(F()) {
        if(Tdash())
            return SUCCESS;
        else
            return FAILED;
    } else
        return FAILED;
}

int Tdash() {
    if(*cursor == '*') {
        cout << cursor << "\t\t T' -> * F T\n";
        cursor++;
        if(F()) {
            if(Tdash())
                return SUCCESS;
            else
                return FAILED;
        } else
            return FAILED;
    } else {
        cout << cursor << "\t\t T' -> $\n";
        return SUCCESS; // T' ->  $\epsilon$  (empty string)
    }
}

```



```

int F() {
    if(*cursor == '(') {
        cout << cursor << "\t\t F -> ( E )\n";
        cursor++;
        if(E()) {
            if(*cursor == ')') {
                cursor++;
                return SUCCESS;
            } else
                return FAILED;
        } else
            return FAILED;
    } else if(*cursor == 'i') {
        cout << cursor << "\t\t F -> i\n";
        cursor++;
        return SUCCESS;
    } else
        return FAILED;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\Compiler Design\" ; if ($?) { g++ recursiveDescent.cpp -o recursiveDescent } ; if ($?) { .\recursiveDescent }
Enter the string: i+i*i

```

Input	Action
i+i*i	E -> T E'
i+i*i	T -> F T'
i+i*i	F -> i
+i*i	T' -> \$
+i*i	E' -> + T E'
i*i	T -> F T'
i*i	F -> i
*i	T' -> * F T'
i	F -> i
	T' -> \$
	E' -> \$

String is successfully parsed

Viva Questions(Program-8)

Question 1:

What is a recursive descent parser?

Answer:

A recursive descent parser is a top-down parser where each non-terminal in the grammar is represented by a recursive function.

Question 2:

How does a recursive descent parser work?

Answer:

It starts from the start symbol and makes recursive calls for each production rule, parsing the input one symbol at a time.

Question 3:

What kind of grammar is suitable for recursive descent parsing?

Answer:

Recursive descent parsing works well with grammars that are non-left-recursive and LL(1), meaning they can be parsed with one lookahead symbol.

Question 4:

What is a lookahead in recursive descent parsing?

Answer:

A lookahead is the next input symbol that the parser uses to decide which production rule to apply.

Question 5:

Can recursive descent parsers handle ambiguous grammars?

Answer:

No, recursive descent parsers cannot handle ambiguous grammars directly. The grammar must be transformed to remove ambiguity.

Program-9.

Aim :- WAP to show all the operations of a stack?

A stack is a linear data structure that follows the Last In First Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed. It operates through two primary operations: push and pop. The push operation adds an element to the top of the stack, while the pop operation removes the element from the top. Stacks can be implemented using arrays or linked lists, with a designated pointer or index to track the top element. Also, stacks often provide a peek operation to view the top element without removing it.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <iostream>

#define MAX 5

using namespace std;

class Stack {
private:
    int stack[MAX];
    int top;
public:
    Stack() : top(-1) {}
    void push(int val) {
        if (top == MAX - 1) {
            cout << "Stack Overflow\n";
            return;
        }
        stack[++top] = val;
        cout << "Pushed " << val << " onto the stack" << endl;
    }
}
```

```

int pop() {
    if(top == -1) {
        cout << "Stack Underflow\n";        return -1;
    }
    cout << "Popped " << stack[top] << " from the stack" << endl;
    return stack[top--];
}

void display() {
    if(top == -1) {
        cout << "Stack is empty\n";        return;
    }
    cout << "Stack elements: ";
    for (int i = 0; i <= top; i++) {        cout << stack[i] << " ";    }
}

};

int main() {
    Stack s;
    s.push(10);        s.push(20);        s.display();
    s.pop();           s.display();        return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS
Code\Compiler Design\" ; if ($?) { g++ Stack.cpp -o Stack } ; if ($?) { .\S
tack }
Pushed 10 onto the stack
Pushed 20 onto the stack
Stack elements: 10 20
Popped 20 from the stack
Stack elements: 10

```

Viva Questions(Program-9)

Question 1:

What are the basic operations of a stack?

Answer:

The basic operations are push, pop, peek (or top), and isEmpty.

Question 2:

What is a stack used for in a compiler?

Answer:

Stacks are used for managing function calls, expression evaluation, and syntax parsing in compilers.

Question 3:

How does the push operation work?

Answer:

Push adds an element to the top of the stack.

Question 4:

What does the pop operation do?

Answer:

Pop removes and returns the element at the top of the stack.

Question 5:

What is the significance of the peek operation?

Answer:

Peek allows viewing the top element of the stack without removing it, useful for decision-making during parsing.

Program-10.

Aim :- WAP to find out the leading of the non-terminals in a grammar?

In a context-free grammar, the leading of a non-terminal refers to the set of terminals that can appear as the first symbols of strings derived from that non-terminal in a single step of a leftmost derivation. The leading is essential for predictive parsing, where the parser decides which production to use based on the next input symbol.

To find the leading of a non-terminal, examine the production rules involving that non-terminal. If the production rule starts with the non-terminal itself or with other non-terminals leading to it, include the first symbols of those production rules in the leading set.

Each procedure corresponds to a non-terminal in the grammar, and the parser tries to match the input with the grammar's production rules by calling these procedures recursively. They read input character by character, consuming terminals as they match against rules, and recursively invoking functions for non-terminals.

Additionally, if the production rule starts with terminals, include them in the leading set. The leading information is crucial for constructing LL(1) parsing tables, ensuring unambiguous parsing with a top-down parser

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

yywrap() : The 'yywrap()' function in a Lex file signals the end of input processing. It is called when 'yylex()' reaches the end of input. By default, 'yywrap()' returns 1 to indicate that no more input files are available. If you need to process multiple files, 'yywrap()' can be customized to handle additional files or perform cleanup tasks. In most simple applications, returning 1 is sufficient to indicate that no further processing is required.

'%%' : In a Lex file, '%%' is used as a delimiter to separate different sections:

- 1. Declarations Section :** Enclosed in '%{ %}', this area includes C code like header files and global variables.
- 2. Rules Section :** The first '%%' separates this section where you define patterns (regular expressions) and their actions.
- 3. User Subroutines Section :** The second '%%' marks the end of the rules section and is where you place additional C functions, such as 'yywrap()' and other helper functions.
- 5. Using these delimiters correctly ensures that Lex can properly parse and generate the necessary code for lexical analysis.**

Source Code

```

#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <string>
#include <sstream>

using namespace std;

void findLeading(const string& nonTerminal, const map<string, vector<string>>&
                productions, set<char>& leading, set<string>& visited) {
    if (visited.count(nonTerminal)) return;
    visited.insert(nonTerminal);
    for (const auto& production : productions.at(nonTerminal)) {
        for (char symbol : production) {
            if (isalpha(symbol)) {
                leading.insert(symbol);
                break; // No need to check further symbols in this production
            } else if (isupper(symbol) && productions.count(string(1, symbol))) {
                findLeading(string(1, symbol), productions, leading, visited);
                break; // Break after checking the first non-terminal
            }
        }
    }
}

int main() {
    map<string, vector<string>> productions;
    productions["A"] = {"BC"};
    productions["B"] = {"a"};
    productions["C"] = {"d"};
    map<string, set<char>> leadingMap;
    for (const auto& production : productions) {

```

```

    const string& nonTerminal = production.first;
    set<char> leading;
    set<string> visited;
    findLeading(nonTerminal, productions, leading, visited);
    leadingMap[nonTerminal] = leading;
}
cout << "Leading of non-terminals:\n";
for (const auto& entry : leadingMap) {
    cout << entry.first << " -> { ";
    for (char terminal : entry.second) {
        cout << terminal << " ";
    }
    cout << "}\n";
}
return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS
Code\Compiler Design\" ; if ($?) { g++ Leading.cpp -o Leading } ; if ($?)
{ .\Leading }
Leading of non-terminals:
A -> { a }
B -> { a }
C -> { d }

```

Viva Questions(Program-10)

Question 1:

What is the leading of a non-terminal?

Answer:

The leading of a non-terminal is the set of terminals that can appear at the beginning of a string derived from that non-terminal.

Question 2:

Why is it important to calculate the leading of non-terminals?

Answer:

It is important for determining which production rule to apply during parsing.

Question 3:

How do you calculate the leading of a non-terminal?

Answer:

By examining the production rules and identifying the terminals that can appear first in each rule.

Question 4:

What is the role of leading sets in predictive parsers?

Answer:

Leading sets help predictive parsers make decisions about which production rule to use based on the current input token.

Question 5:

What is the difference between leading and first sets?

Answer:

Leading refers to the terminals that appear at the start of a derivation, while first is used to refer to the first terminal that can be derived from a non-terminal.

Program-11.

Aim :- WAP to Implement Shift Reduce parsing for a String ?

In Compiler , shift-reduced parser is a type of bottom-up parser used in compiler design to check the syntax of input strings based on context-free grammars. The parsing process involves two main actions: "shift," which moves the next input symbol onto a stack, and "reduce," which replaces a sequence of symbols on the stack with a non-terminal symbol according to the production rules of the grammar.

This approach efficiently handles ambiguous grammars and is commonly implemented using algorithms like LR parsing, which allows for efficient parsing of programming languages by determining the next action based on a state and the current input symbol.

Steps in Shift-Reduce Parsing :-

1. **Shift:** Move the next input symbol onto the stack.
2. **Reduce:** Replace the top of the stack with a non-terminal based on the production rules.
3. **Accept:** If the entire input has been consumed and the stack contains only the start symbol of the grammar, the parsing is successful.
4. **Error:** If neither action can be taken, report an error

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
vector<char> st , a;
void StackAndInput() {
    for (auto x : st){      cout << x<< "\t";      }
    for (auto x : a) {      cout << x;              }
}
```



```

void check() {
    for (int i = 0; i < st.size(); i++) {
        if (st[i] == 'a') {
            st[i] = 'E';
            StackAndInput();
            cout << "$\t" << "REDUCE E->a";
            check();
        }
        if (i + 2 < st.size() && st[i] == 'E' && (st[i + 1] == '+' || st[i + 1] == '*') st[i + 2] == 'E') {
            st.pop_back();      st.pop_back();
            StackAndInput();
            if (st[i + 1] == '+') {
                cout << "$\t" << "REDUCE E->E+E";
            } else if (st[i + 1] == '*') {
                cout << "$\t" << "REDUCE E->E*E";
            }
        }
    }
}

int main() {
    cout << "GRAMMAR is: \nE -> E + E \nE -> E * E \nE -> a\n";
    string input;
    cout << "Enter input string (consisting of 'a', '+', '*'): ";
    cin >> input;
    for (char ch : input) {
        a.push_back(ch);
    }
}

```

```

cout << "\nStack\tInput\tAction";
for (int i = 0; i < a.size(); i++) {
    st.push_back(a[i]);
    a[i] = ' ';
    StackAndInput();
    cout << "$\tSHIFT -> " << st.back();
    check();
}
if (st.size() == 1 && st[0] == 'E') {
    cout << "\n\nString accepted";
} else {
    cout << "\n\nString rejected";
}
return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ shiftred.cpp -o shiftred } ; if ($?) { .\shif
red }
GRAMMAR is:
E -> E + E
E -> E * E
E -> a
Enter input string (consisting of 'a', '+', '*'): a+a*a

Stack   Input   Action
$a      +a*a$    SHIFT -> a
$E      +a*a$    REDUCE E->a
$E+     a*a$     SHIFT -> +
$E+a    *a$      SHIFT -> a
$E+E    *a$      REDUCE E->a
$E      *a$     REDUCE E->E+E
$E*     a$      SHIFT -> *
$E*a    $       SHIFT -> a
$E*E    $       REDUCE E->a
$E      $       REDUCE E->E*E

String accepted

```

Viva Questions(Program-11)

Question 1:

What is shift-reduce parsing?

Answer:

Shift-reduce parsing is a bottom-up parsing technique that uses a stack to shift input tokens and reduce them according to production rules.

Question 2:

What is the “shift” operation in shift-reduce parsing?

Answer:

The “shift” operation moves the next input symbol onto the parsing stack.

Question 3:

What is the “reduce” operation in shift-reduce parsing?

Answer:

The “reduce” operation replaces a sequence of symbols on the stack with a non-terminal, based on a production rule.

Question 4:

What is a handle in shift-reduce parsing?

Answer:

A handle is the substring that matches the right-hand side of a production rule, which is then reduced to a non-terminal.

Question 5:

What are the common types of conflicts in shift-reduce parsing?

Answer:

Shift-reduce conflicts and reduce-reduce conflicts occur when the parser cannot decide whether to shift or reduce.

Program-12.

Aim :- WAP to find out the FIRST of the Non-terminals in a grammar?

In Compiler , the FIRST of a non-terminal refers to the set of terminal symbols that can appear at the beginning of any string derived from that non-terminal. It helps in constructing parsing tables, especially for LL and LR parsers, by determining which terminal symbol should be expected first during parsing.

To compute the FIRST set, we examine the production rules for the non-terminal and recursively explore the possible terminals or non-terminals on the right-hand side of each rule. If a non-terminal can derive an empty string (ϵ), the FIRST set includes ϵ , indicating that the non-terminal can be followed by other symbols in certain cases.

Rules to compute FIRST set :-

1. If x is a terminal, then $\text{FIRST}(x) = \{ 'x' \}$
2. If $x \rightarrow ?$, is a production rule, then add ? to $\text{FIRST}(x)$.
3. If $X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$ is a production,
 - $\text{FIRST}(X) = \text{FIRST}(Y_1)$
 - If $\text{FIRST}(Y_1)$ contains ? then $\text{FIRST}(X) = \{ \text{FIRST}(Y_1) - ? \} \cup \{ \text{FIRST}(Y_2) \}$
 - If $\text{FIRST}(Y_i)$ contains ? for all $i = 1$ to n , then add ? to $\text{FIRST}(X)$.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <bits/stdc++.h>

using namespace std;

char productionSet[10][10];

void FIRST(set<char> &result, char c) {
    if(!(isupper(c))) {    result.insert(c);    return;    }
    for (int i = 0; i < n; i++) {
        if(productionSet[i][0] == c) {    FIRST(result, productionSet[i][2]);    }
    }
}
```

```

int main() {
    cin >> n;
    set<char> s;
    for (int i = 0; i < n; i++) {
        cin >> productionSet[i];
        s.insert(productionSet[i][0]);
    }
    for (auto c : s) {
        set<char> result;
        FIRST(result, c);
        cout << "First (" << c << ") : { ";
        for (auto x : result) {
            cout << x << " ";
        }
        cout << " }\n";
    }
    return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ First.cpp -o First } ; if ($?) { .\First }
8
S=AB
B=aAB
B=b
A=CD
D=cCD
D=b
C=(E)
C=d
First (A) : { ( d }
First (B) : { a b }
First (C) : { ( d }
First (D) : { b c }
First (S) : { ( d }

```


Viva Questions(Program-12)

Question 1:

What is the FIRST set in a grammar?

Answer:

The FIRST set of a non-terminal contains the terminals that can appear at the beginning of a string derived from that non-terminal.

Question 2:

How is the FIRST set used in predictive parsing?

Answer:

The FIRST set helps the parser determine which production rule to apply based on the next input symbol.

Question 3:

How do you calculate the FIRST set for a non-terminal?

Answer:

By recursively analyzing the production rules and collecting the first terminal symbols that appear in each derivation.

Question 4:

Can ϵ (epsilon) be part of the FIRST set?

Answer:

Yes, if a non-terminal can derive an empty string (ϵ), then ϵ is included in its FIRST set.

Question 5:

How do you handle FIRST sets for non-terminals that start with other non-terminals?

Answer:

The FIRST set of the non-terminal includes the FIRST set of the non-terminal that appears at the beginning of its production rules.

Program-13.

Aim :- WAP to check whether a grammar is operator precedent.?

In Compiler , Operator precedence parsing is a type of Shift Reduce Parsing. In operator precedence parsing, an operator grammar and an input string are fed as input to the operator precedence parser, which may generate a parse tree. Note that a parse tree will only be generated if the input string gets accepted.

In operator precedence parsing, the shift and reduce operations are done based on the priority between the symbol at the top of the stack and the current input symbol. The operator precedence parser performs the operator precedence parsing using operator grammar.

A grammar is said to be an operator grammar if it follows these two properties:

1. There should be no ϵ (epsilon) on the right-hand side of any production.
2. There should be no two non-terminals adjacent to each other. Examples of operator grammar are $A + B$, $A - B$, $A \times B$, etc

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <iostream>

#include <vector>

#include <string>

using namespace std;

void exitWithError() {

    cout << "Not operator grammar" << endl;    exit(0);

}

int main() {

    int n;

    cout << "Enter the number of productions: ";    cin >> n;

    vector<string> grammar(n);

    cout << "Enter the productions (e.g., A=B+C):\n";
```

```

for (int i = 0; i < n; i++) {      cin >> grammar[i];    }
for (const auto& prod : grammar) {
    int j = 2;      char c;
    while (prod[j] != '\0') {
        c = prod[j];
        if (c == '+' || c == '-' || c == '*' || c == '/') {
            if (prod[j + 1] == '\0' || prod[j + 1] == '$') {          exitWithError();          }
        } else if (c == '$') {
            exitWithError();
        } else if (!(isupper(c)) && c != 'A') {
            exitWithError();
        }      j++;
    }
}
cout << "Operator grammar" << endl;
return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ OperatorPre.cpp -o OperatorPre } ; if ($?) { .
\OperatorPre }
Enter the number of productions: 3
Enter the productions (e.g., A=B+C):
A=A*A
B=AA
A=$
Not operator grammar
PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ OperatorPre.cpp -o OperatorPre } ; if ($?) { .
\OperatorPre }
Enter the number of productions: 2
Enter the productions (e.g., A=B+C):
A=A/A
B=A+A
Operator grammar

```

Viva Questions(Program-13)

Question 1:

What is an operator-precedence grammar?

Answer:

An operator-precedence grammar allows the parser to determine the precedence of operators without needing to transform the grammar.

Question 2:

What are precedence relations in an operator-precedence grammar?

Answer:

Precedence relations define whether one operator has higher, lower, or equal precedence compared to another.

Question 3:

How do you check if a grammar is operator-precedence?

Answer:

By constructing the precedence relations between terminals and ensuring that the grammar has no ambiguity or conflicts.

Question 4:

What is the difference between a shift-reduce parser and an operator-precedence parser?

Answer:

An operator-precedence parser specifically uses precedence relations to resolve conflicts, while shift-reduce parsers rely on general production rules.

Question 5:

Can all grammars be transformed into operator-precedence grammars?

Answer:

No, only grammars that do not contain ambiguities or certain types of recursion can be transformed into operator-precedence grammars.

Program-14.

Aim :- WAP to implement Intermediate Code Generation (ICG) for simple expressions In The Compiler ?

In Compiler , Intermediate Code Generation is a stage in the process of compiling a program, where the compiler translates the source code into an intermediate representation. This representation is not machine code but is simpler than the original high-level code. Intermediate Code Generation in a compiler involves translating high-level source code into an intermediate representation (IR) that is independent of machine architecture. This IR serves as an abstraction between the source code and the target machine code, making optimization and portability easier And More Understandable By The Compiler .

Here's how it works:

- **Translation:** The compiler takes the high-level code (like C or Java) and converts it into an intermediate form, which can be easier to analyse and manipulate.
- **Portability:** This intermediate code can often run on different types of machines without needing major changes, making it more versatile.
- **Optimization:** Before turning it into machine code, the compiler can optimize this intermediate code to make the final program run faster or use less memory.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <iostream>
```

```
#include <sstream>
```

```
#include <vector>
```

```
#include <string>
```

```
#include <map>
```

```
using namespace std;
```

```
void generateIntermediateCode(const string& expression) {
```

```
    stringstream ss(expression);
```

```

string token;
vector<string> tokens;

while (ss >> token) {
    tokens.push_back(token);
}

int tempCount = 1;
vector<string> intermediateCode;
vector<string> operands;
vector<string> operators;

for (const auto& t : tokens) {
    if (t == "=") {
        continue;
    } else if (t == "+" || t == "-" || t == "*" || t == "/") {
        operators.push_back(t);
    } else {
        operands.push_back(t);
        if (operators.size() > 0) {
            string op = operators.back();
            operators.pop_back();
            string rhs2 = operands.back(); operands.pop_back();
            string rhs1 = operands.back(); operands.pop_back();
            string tempVar = "t" + to_string(tempCount++);
            intermediateCode.push_back(tempVar + " = " + rhs1 + " " + op + " " + rhs2);
            operands.push_back(tempVar);
        }
    }
}

```



```

    if(!operands.empty()) {
        string lhs = tokens[0];
        string finalResult = operands.back();
        intermediateCode.push_back(lhs + " = " + finalResult);
    }
    cout << "Intermediate Code:\n";
    for (const auto& code : intermediateCode) {
        cout << code << endl;
    }
}

int main() {
    string expression;
    cout << "Enter an expression (e.g., a = b + c * d): ";
    getline(cin, expression);
    generateIntermediateCode(expression);
    return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ IntermediateCode.cpp -o IntermediateCode } ; i
f ($?) { .\IntermediateCode }
Enter an expression (e.g., a = b + c * d): a = b + c * d
Intermediate Code:
t1 = b + c
t2 = t1 * d
a = t2

```

Viva Questions(Program-14)

Question 1:

What is intermediate code in compiler design?

Answer:

Intermediate code is a low-level, machine-independent representation of the source program, typically generated between the source code and target code.

Question 2:

Why do we generate intermediate code?

Answer:

Intermediate code simplifies optimization and allows compilers to target multiple machine architectures.

Question 3:

What is three-address code?

Answer:

Three-address code is a type of intermediate code where each instruction contains at most three operands, typically for binary operations.

Question 4:

What is a quadruple in intermediate code?

Answer:

A quadruple is a representation of an intermediate code instruction with four fields: operator, argument1, argument2, and result.

Question 5:

How do you generate intermediate code for an arithmetic expression?

Answer:

By breaking the expression into smaller parts and assigning temporary variables to store intermediate results.

Program-15.

Aim :- WAP to minimize any given DFA?

In Compiler , DFA stands for Deterministic Finite Automaton. Its job is to read in a string of symbols (input) and move (transition) between its states based on the input and ultimately accept or reject the string of symbols. The transitions are deterministic: for a given symbol string and a given DFA, the DFA will go through the same states every time the same symbol string is fed into it. In other words, the path between the states is guaranteed to be unique for a given symbol string. The way this is enforced is by specifying that for every state, for a given input symbol, there's only one transition.

Here are the steps to minimize a DFA (Deterministic Finite Automaton):

1. **Remove unreachable states:** Eliminate any states that can't be reached from initial state.
2. **Distinguish final and non-final states:** Partition the states into two groups: final states and non-final states.
3. **Refine state partitions:** Iteratively partition the states further based on their transitions. States are considered equivalent if, for every input symbol, transition to same partition.
4. **Merge equivalent states:** Combine all equivalent states into a single state to create the minimized DFA.
5. **Construct the minimized DFA:** Define the new DFA with minimized states and update transitions accordingly.

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <bits/stdc++.h>

using namespace std;

const int N = 109;

int t[N][N];

bool fs[N];

bool printed[N];

vector<vector<int>>> pi[2];

int n, m, nfs;
```

```

int idx(int x) {
    for (int i = 0; i < pi[0].size(); i++) {
        for (int j = 0; j < pi[0][i].size(); j++) {
            if (pi[0][i][j] == x) {
                return i;
            }
        }
    }
    return -1;
}

void print_min() {
    cout << "\nMinimized DFA Table:\n";
    cout << "-----\n";
    cout << "Q\t";
    for (int j = 0; j < m; j++) {
        cout << j << "\t";
    }
    cout << endl << endl;
    for (int i = 0; i < n; i++) {
        if (printed[i]) continue;
        int ind = idx(i);
        cout << "[";
        for (int k = 0; k < pi[0][ind].size(); k++) {
            cout << pi[0][ind][k];
            printed[pi[0][ind][k]] = true;
            if (k != pi[0][ind].size() - 1) cout << ",";
        }
        cout << "]\t";
        for (int j = 0; j < m; j++) {
            ind = idx(t[i][j]);

```

```

        cout << "[";
        for (int k = 0; k < pi[0][ind].size(); k++) {
            cout << pi[0][ind][k];
            if (k != pi[0][ind].size() - 1) cout << ",";
        }
        cout << "]\t";
    }
    cout << endl;
}
cout << endl;
}

int main() {
    cout << "Enter number of states, input symbols, and final states: ";
    cin >> n >> m >> nfs;
    cout << "Enter the final states: ";
    for (int i = 0; i < nfs; i++) {
        int x;
        cin >> x;
        fs[x] = true;
    }
    cout << "Enter the transition table (state x input -> next state):\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cin >> t[i][j];
        }
    }
    vector<int> v[2];
    for (int i = 0; i < n; i++) {
        v[fs[i]].push_back(i);
    }
}

```

```

pi[0].push_back(v[0]);
pi[0].push_back(v[1]);
pi[1] = pi[0];
int iters = 100;
while (iters--) {
    pi[0] = pi[1];
    for (int i = 0; i < pi[1].size(); i++) {
        for (int j = 1; j < pi[1][i].size(); j++) {
            bool placed = false;
            for (int k = 0; k < j; k++) {
                bool compatible = true;
                for (int l = 0; l < m; l++) {
                    if (idx(t[pi[1][i][j]][l]) != idx(t[pi[1][i][k]][l])) {
                        compatible = false;
                        break;
                    }
                }
            }
            if (compatible) {
                placed = true;
                break;
            }
        }
        if (!placed) {
            bool found = false;
            for (int x = pi[0].size(); x < pi[1].size(); x++) {
                for (int k = 0; k < pi[1][x].size(); k++) {
                    bool compatible = true;
                    for (int l = 0; l < m; l++) {
                        if (idx(t[pi[1][i][j]][l]) != idx(t[pi[1][x][k]][l])) {
                            compatible = false;

```



```

        break;
    }
}
if (compatible) {
    pi[1][x].push_back(pi[1][i][j]);
    pi[1][i].erase(pi[1][i].begin() + j);
    placed = true;
    found = true;
    break;
}
}
if (found) break;
}
if (!found) {
    vector<int> newClass = {pi[1][i][j]};
    pi[1].push_back(newClass);
    pi[1][i].erase(pi[1][i].begin() + j);
}
}
}
}

for (int i = 0; i < pi[0].size(); i++) {
    sort(pi[0][i].begin(), pi[0][i].end());
}
sort(pi[0].begin(), pi[0].end());
for (int i = 0; i < pi[1].size(); i++) {
    sort(pi[1][i].begin(), pi[1][i].end());
}
sort(pi[1].begin(), pi[1].end());

```

```

        if(pi[0] == pi[1]) break;
    }
    assert(pi[0] == pi[1]);
    print_min();
    return 0;
}

```

OUTPUT : -

```

PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ DFAMin.cpp -o DFAMin } ; if ($?) { .\DFAMin }
Enter number of states, input symbols, and final states: 5 2 1
Enter the final states: 4
Enter the transition table (state x input -> next state):
1 2
1 3
1 2
1 4
1 2

Minimized DFA Table:
-----
Q      0      1
[0,2]  [1]    [0,2]
[1]    [1]    [3]
[3]    [1]    [4]
[4]    [1]    [0,2]

```

Viva Questions(Program-15)

Question 1:

What is DFA minimization?

Answer:

DFA minimization is the process of reducing the number of states in a deterministic finite automaton (DFA) while preserving its language.

Question 2:

Why is DFA minimization important?

Answer:

It simplifies the DFA, making it more efficient in terms of time and space during execution.

Question 3:

How do you minimize a DFA?

Answer:

By identifying and merging equivalent states, typically using techniques like partition refinement.

Question 4:

What are equivalent states in a DFA?

Answer:

Equivalent states are states that behave identically for all possible input strings, leading to the same set of transitions.

Question 5:

Can a minimized DFA be further reduced?

Answer:

No, once a DFA is minimized, it cannot be further reduced without altering the language it accepts.

Program-16.

Aim :- WAP to convert NFA to DFA?

In Compiler , An Non-deterministic Finite Automaton (NFA) is a type of finite state machine where for each state and input symbol, there may be multiple possible next states or no transition at all. Unlike a DFA, an NFA can have epsilon (ϵ) transitions, allowing state changes without consuming an input symbol.

DFA stands for Deterministic Finite Automaton. Its job is to read in a string of symbols (input) and move (transition) between its states based on the input and ultimately accept or reject the string of symbols. The transitions are deterministic: for a given symbol string and a given DFA, the DFA will go through the same states every time .

Here are the steps to convert an NFA to a DFA:

1. Start state: Begin with the ϵ -closure of the NFA's start state as the DFA's start state.
2. Transitions: For each DFA state, compute its transitions by considering all possible NFA states reachable via each input symbol.
3. New states: Add new DFA states for any sets of NFA states not yet represented in the DFA.
4. Final states: Any DFA state that includes an NFA final state becomes a final state in the DFA.

❖ **Repeat the process until no new DFA states are generated.**

(.l) File : A '.l' file is a **source file used in lexical analysis**, typically associated with the Flex tool (Fast Lexical Analyzer). It contains rules and patterns that define how input text should be tokenized or broken down into meaningful symbols, known as tokens.

(.y) File : A '.y' file is a source file for **Yacc (Yet Another Compiler Compiler)**, which defines the grammar of a programming language using BNF notation. It contains rules for parsing tokens generated by a lexer, specifying how to construct parse trees and perform actions during parsing, crucial for compiler design.

Source Code

```
#include <bits/stdc++.h>

using namespace std;

void print(vector<vector<vector<int>>> table) {
    cout << " STATE/INPUT |";
    char a = 'a';
    for (int i = 0; i < table[0].size() - 1; i++) {
        cout << " " << a++ << " |";
    }
}
```

```

cout << " ^ " << endl << endl;
for (int i = 0; i < table.size(); i++) {
    cout << " " << i << " ";
    for (int j = 0; j < table[i].size(); j++) {
        cout << " | ";
        for (int k = 0; k < table[i][j].size(); k++) {
            cout << table[i][j][k] << " ";
        }
        if (table[i][j].empty()) {            cout << "^ ";            }
    }    cout << endl;
}
}

void printdfa(vector<vector<int>> states, vector<vector<vector<int>>> dfa) {
    cout << "\n STATE/INPUT ";
    char a = 'a';
    for (int i = 0; i < dfa[0].size(); i++) {
        cout << "| " << a++ << " ";
    }
    cout << endl;
    for (int i = 0; i < states.size(); i++) {
        cout << "{ ";
        for (int h = 0; h < states[i].size(); h++)
            cout << states[i][h] << " ";
        if (states[i].empty()) {
            cout << "^ ";
        }
        cout << "} ";
        for (int j = 0; j < dfa[i].size(); j++) {
            cout << " | ";
            for (int k = 0; k < dfa[i][j].size(); k++) {

```

```

        cout << dfa[i][j][k] << " ";
    }
    if (dfa[i][j].empty()) {        cout << "^ ";        }
}    cout << endl;
}
}

vector<int> closure(int s, vector<vector<vector<int>>> v) {
    vector<int> t;
    queue<int> q;
    t.push_back(s);
    int a = v[s][v[s].size() - 1].size();
    for (int i = 0; i < a; i++) {
        t.push_back(v[s][v[s].size() - 1][i]);
        q.push(t.back());
    }
    while (!q.empty()) {
        int f = q.front();
        q.pop();
        if (!v[f][v[f].size() - 1].empty()) {
            int u = v[f][v[f].size() - 1].size();
            for (int i = 0; i < u; i++) {
                int y = v[f][v[f].size() - 1][i];
                if (find(t.begin(), t.end(), y) == t.end()) {
                    t.push_back(y);
                    q.push(y);
                }
            }
        }
    }
    return t;
}

```



```

int main() {
    int n, alpha;

    cout << "***** NFA to DFA *****";

    cout << "Enter total number of states in NFA: ";

    cin >> n;

    cout << "Enter number of elements in alphabet: ";

    cin >> alpha;

    vector<vector<vector<int>>> table;

    for (int i = 0; i < n; i++) {
        cout << "For state " << i << endl;

        vector<vector<int>> v;

        char a = 'a';

        int y, yn;

        for (int j = 0; j < alpha; j++) {
            vector<int> t;

            cout << "Enter number of output states for input " << a++ << ": ";

            cin >> yn;

            cout << "Enter output states: " << endl;

            for (int k = 0; k < yn; k++) {
                cin >> y;

                t.push_back(y);
            }

            v.push_back(t);
        }

        vector<int> t;

        cout << "Enter number of output states for input ^ : ";

        cin >> yn;

        cout << "Enter output states: " << endl;

        for (int k = 0; k < yn; k++) {
            cin >> y;

```

```

        t.push_back(y);
    }
    table.push_back(v);
}
cout << "***** TRANSITION TABLE OF NFA *****" << endl;
print(table);
cout << endl << "***** TRANSITION TABLE OF DFA *****" << endl;
vector<vector<vector<int>>>> dfa;
vector<vector<int>> states;
states.push_back(closure(0, table));
queue<vector<int>> q;
q.push(states[0]);
while (!q.empty()) {
    vector<int> f = q.front();
    q.pop();
    vector<vector<int>> v;
    for (const auto& state : s) {
        t.push_back(state);
    }
    v.push_back(t);
    if (find(states.begin(), states.end(), t) == states.end()) {
        states.push_back(t);
        q.push(t);
    }
}
dfa.push_back(v);
}
printdfa(states, dfa);
return 0;
}

```

OUTPUT :-

```
PS C:\Users\HP\Desktop\VS Code\Compiler Design> cd "c:\Users\HP\Desktop\VS Code\
Compiler Design\" ; if ($?) { g++ nfa_dfa.cpp -o nfa_dfa } ; if ($?) { .\nfa_dfa
}
***** NFA to DFA *****

Enter total number of states in NFA: 3
Enter number of elements in alphabet: 2
For state 0
Enter number of output states for input a: 1
Enter output states:
1
Enter number of output states for input b: 2
Enter output states:
0 2
Enter number of output states for input ^ : 0
Enter output states:
For state 1
Enter number of output states for input a: 1
Enter output states:
1
Enter number of output states for input b: 1
Enter output states:
2
Enter number of output states for input ^ : 1
Enter output states:
2
For state 2
Enter number of output states for input a: 0
Enter output states:
Enter number of output states for input b: 1
Enter output states:
0
Enter number of output states for input ^ : 0
Enter output states:
***** TRANSITION TABLE OF NFA *****
STATE/INPUT | a | b | ^
-----
0 | 1 | 0 2 | ^
1 | 1 | 2 | 2
2 | ^ | 0 | ^

***** TRANSITION TABLE OF DFA *****
STATE/INPUT | a | b
-----
{ 0 } | 1 2 | 0 2
{ 1 2 } | 1 2 | 0 2
{ 0 2 } | 1 2 | 0 2
```

Viva Questions(Program-16)

Question 1:

What is the difference between an NFA and a DFA?

Answer:

An NFA (Non-deterministic Finite Automaton) allows multiple transitions for the same input symbol or ϵ -transitions, while a DFA (Deterministic Finite Automaton) has exactly one transition per input symbol.

Question 2:

What is the subset construction method?

Answer:

The subset construction method is used to convert an NFA into an equivalent DFA by creating states in the DFA that represent sets of NFA states.

Question 3:

Why do we convert an NFA to a DFA?

Answer:

A DFA is easier to implement as it guarantees a single unique transition for each input symbol, which simplifies pattern recognition.

Question 4:

Does every NFA have an equivalent DFA?

Answer:

Yes, every NFA can be converted into an equivalent DFA that accepts the same language.

Question 5:

How does the state explosion problem affect NFA to DFA conversion?

Answer:

The number of states in the resulting DFA can grow exponentially compared to the NFA, leading to the state explosion problem.